

Clasificador de Clima con K-Nearest Neighbors (KNN)

Syllabus Académico Oficial — Mes 01 / 12

Ruta de Formación IA/ML · Fase 1: Fundamentos ML · Mayo 2026

Código: **IA-ML-01**

Nivel: Undergraduate — Introducción a ML

Carga: 8–10 h/semana · 4 semanas

Mayo 2026

Prerrequisitos

Python intermedio

NumPy básico · Manejo de CSV

Stack Tecnológico

Python, NumPy, Pandas

Matplotlib, scikit-learn, Jupyter

Resumen—Este syllabus define los objetivos, contenidos, metodología y criterios de evaluación del primer proyecto de la ruta de formación en Inteligencia Artificial y Machine Learning de 12 meses. El estudiante implementa un clasificador de condiciones climáticas (soleado, nublado, lluvia, nieve) utilizando el algoritmo K-Nearest Neighbors (KNN) sobre el dataset UCI Weather AUS. El proyecto cubre el ciclo completo de clasificación supervisada: análisis exploratorio de datos (EDA), preprocesamiento, implementación manual de KNN en Python puro, comparación con scikit-learn, y evaluación con métricas estándar. Los objetivos de aprendizaje están organizados según la Taxonomía de Bloom revisada (Recordar–Crear) [2] y conectados explícitamente con las competencias requeridas en el proyecto de investigación CausalBench (Mes 12).

Index Terms—K-Nearest Neighbors, Clasificación Supervisada, Machine Learning, Taxonomía de Bloom, Análisis Exploratorio de Datos, UCI Weather AUS, Bias-Variance Tradeoff, CausalBench

I. IDENTIFICACIÓN DEL CURSO

Cuadro I
FICHA DE IDENTIFICACIÓN — IA-ML-01

Campo	Descripción
Nombre	Clasificador de Clima con KNN
Código	IA-ML-01 · Mes 01 / 12
Nivel	Undergraduate — Introducción a ML
Créditos	1 crédito académico (≈ 36 h totales)
Carga	8–10 h/semana · 4 semanas
Modalidad	Autónoma con revisión entre pares
Dataset	UCI Weather AUS (<i>multi-clase</i>)
Posición	Proyecto 01/12 → conecta con Mes 02

Prerrequisitos. Python intermedio (listas, funciones, clases básicas); NumPy básico; manejo de archivos CSV con Pandas; álgebra vectorial elemental (producto punto, norma).

II. DESCRIPCIÓN GENERAL

Este curso constituye el punto de entrada a la ruta de formación de 12 meses. A través del desarrollo de un clasificador

climático con KNN, el estudiante adquiere los fundamentos metodológicos del aprendizaje supervisado: carga y limpieza de datos, entrenamiento, evaluación y visualización de resultados.

El proyecto tiene **doble propósito**: (1) dominar las mecánicas matemáticas de KNN implementándolo desde cero en Python puro, y (2) comparar dicha implementación con la versión optimizada de scikit-learn [4]. Al finalizar, el estudiante produce su primer entregable en **formato académico IEEE** y establece hábitos de documentación profesional que se mantienen durante los 11 meses siguientes.

III. ONTOLOGÍA DEL DOMINIO

La Tabla II define los conceptos centrales del dominio y sus relaciones jerárquicas. Las relaciones estructurales clave son:

- *Es-un*: KNN es un algoritmo de clasificación supervisada.
- *Usa*: KNN usa una métrica de distancia para calcular similitud.
- *Requiere*: KNN requiere normalización de features para operar correctamente.
- *Determina*: El hiperparámetro K determina el suavizado de la frontera de decisión.
- *Produce*: El pipeline produce una predicción de clase climática a partir de variables meteorológicas.

IV. OBJETIVOS DE APRENDIZAJE — TAXONOMÍA DE BLOOM

Los objetivos de aprendizaje están organizados según la Taxonomía de Bloom Revisada [2] (Anderson & Krathwohl, 2001), de menor a mayor complejidad cognitiva. La Tabla III sintetiza los seis niveles.

IV-A. Detalle por Nivel

Nivel 1 — Recordar

- **OA-1.1** Definir aprendizaje supervisado, clasificación y KNN (no-paramétrico, lazy learner, sensible a escala).
- **OA-1.2** Identificar las métricas de distancia (Euclidiana, Manhattan, Minkowski) y enunciar cuándo aplica cada una.

Cuadro II
ONTOLOGÍA DEL DOMINIO — CLASIFICACIÓN SUPERVISADA CON KNN

Nivel	Concepto	Definición operacional
Dominio	ML Supervisado	Paradigma donde el modelo aprende de pares (entrada, etiqueta).
Subdominio	Clasificación	Asignar una instancia a una categoría discreta predefinida.
Algoritmo	KNN	Clasificador no-paramétrico basado en similitud con los k vecinos más cercanos.
Espacio	Feature space	Representación vectorial del conjunto de variables de entrada.
Métrica	Dist. Euclid.	$d(\mathbf{x}, \mathbf{y}) = (\sum_i (x_i - y_i)^2)^{1/2}$
Dato	Instancia	Registro $\{T, H, P\}$ con etiqueta de clima.
Proceso	Preproc.	Transformación de datos crudos a representación apta.
Evaluación	Accuracy / CM	Métricas para medir la calidad predictiva del clasificador.
Hiperpar.	K	Parámetro externo que controla la complejidad del modelo.
Fenómeno	Bias-Variance	Tensión entre sesgo (underfitting) y varianza (overfitting) según k .

Cuadro III
OBJETIVOS DE APRENDIZAJE POR NIVEL BLOOM

Nv.	Bloom	Verbo	Objetivo / Evidencia
1	Recordar	Definir / Identificar	Conceptos KNN y métricas de distancia. <i>Ev.: glosario de 20 términos.</i>
2	Comprender	Explicar / Interpretar	Efecto de K en bias-variance; interpretar matriz de confusión. <i>Ev.: sección teórica del reporte.</i>
3	Aplicar	Implementar / Ejecutar	KNN manual y pipeline completo EDA→modelo→eval. <i>Ev.: notebook reproducible.</i>
4	Analizar	Comparar / Diferenciar	Impl. propia vs. sklearn; Min-Max vs. Z-score. <i>Ev.: tabla comparativa.</i>
5	Evaluar	Juzgar	Límites de KNN (dimensionalidad, tamaño de datos). <i>Ev.: sección de conclusiones.</i>
6	Crear	Diseñar	Pipeline reutilizable documentado. <i>Ev.: repositorio GitHub con README.</i>

Nivel 2 — Comprender

- **OA-2.1** *Explicar* el efecto de K en el bias-variance tradeoff

con ejemplos del dataset climático.

- **OA-2.2** *Interpretar* la matriz de confusión para identificar clases problemáticas (e.g., nublado vs. lluvia).

Nivel 3 — Aplicar

- **OA-3.1** *Implementar* KNN desde cero en Python puro (sin sklearn), incluyendo cálculo de distancias y votación por mayoría.
- **OA-3.2** *Ejecutar* el pipeline completo: carga CSV → EDA → normalización → train/test split → KNN → evaluación.

Nivel 4 — Analizar

- **OA-4.1** *Comparar* implementación propia vs. sklearn en accuracy, tiempo de ejecución (`timeit`) y uso de memoria.
- **OA-4.2** *Diferenciar* los efectos de normalización Min-Max vs. estandarización Z-score sobre las fronteras de decisión de KNN.

Nivel 5 — Evaluar

- **OA-5.1** *Juzgar* cuándo KNN es apropiado vs. inapropiado según dimensionalidad (curse of dimensionality), tamaño del dataset y distribución de clases.

Nivel 6 — Crear

- **OA-6.1** *Diseñar* y documentar un pipeline reutilizable adaptable a nuevos datasets con mínimos cambios de configuración.

V. CONTENIDO TEMÁTICO — SEMANA A SEMANA

V-A. Semana 1 — Marco Teórico y Fundamentos Matemáticos

Objetivo: Establecer el fundamento conceptual y matemático necesario para implementar KNN con comprensión profunda.

Cuadro IV
CONTENIDOS — SEMANA 1

#	Tema	Subtemas	Recurso
1.1	Paradigmas ML	Supervisado, no sup., reforzado. Flujo general.	Gérón [1] Cap. 1
1.2	Espacios y dist.	Vectores, d_E , d_M , d_p . Impacto de escala.	NumPy docs
1.3	Intuición KNN	Vecindad, votación, $O(n \cdot d)$.	Gérón Cap. 3
1.4	Bias-Variance	Underfitting (K grande) vs. overfitting (K peq.)	StatQuest [6]
1.5	UCI Weather AUS	Variables, distribución de clases.	UCI [3]

V-B. Semana 2 — Implementación y Preprocesamiento

Objetivo: Implementar KNN desde cero y establecer el pipeline de preprocesamiento.

Cuadro V
CONTENIDOS — SEMANA 2

#	Tema	Subtemas	Recurso
2.1	EDA	Estadísticos, outliers (IQR), correlación.	Pandas [5]
2.2	Preproc.	Imputación, codificación, split 80/20.	sklearn [4]
2.3	Normalización	Min-Max vs. Z-score. Impacto en KNN.	sklearn.preprocessing
2.4	KNN manual	Clase: <code>fit()</code> , <code>predict()</code> , <code>votar()</code> .	Python OOP
2.5	Validación	Hold-out, K-Fold, por qué un split engaña.	sklearn.model_selection

Cuadro VII
CONTENIDOS — SEMANA 4

#	Tema	Subtemas	Recurso
4.1	Escritura IEEE	Abstract, Intro, Dataset, Method, Results, Concl.	IEEE Template
4.2	Análisis res.	Estadística, errores, limitaciones.	Gérón Cap. 3
4.3	Doc. código	Docstrings (NumPy style), README, requirements.	PEP 257
4.4	Conex. Causal.	Pipeline de clasificación → CausalBench.	Roadmap IA/ML
4.5	Retrospectiva	Gaps, preview Mes 02.	Notion / Obsidian

V-C. Semana 3 — Experimentación, Comparación y Visualización

Objetivo: Ejecutar experimentos sistemáticos, comparar implementaciones y generar visualizaciones de análisis.

- CT-03** Aplicar técnicas de normalización y analizar su impacto en KNN.
- CT-04** Visualizar fronteras de decisión y matrices de confusión.
- CT-05** Comparar implementaciones propias con librerías optimizadas.

Cuadro VI
CONTENIDOS — SEMANA 3

#	Tema	Subtemas	Recurso
3.1	Métricas	Accuracy, Precisión, Recall, F1 macro/micro.	sklearn.metrics
3.2	Matriz conf.	Interpretación visual, clases problemáticas.	seaborn
3.3	Curva K -Acc.	Sweep $K = 1 \dots 30$, K^* por validación.	Matplotlib
3.4	Fronteras 2D	Reducción 2 features, meshgrid, regiones.	numpy.meshgrid
3.5	Comparación	Accuracy, <code>timeit</code> , legibilidad vs. sklearn.	timeit

V-D. Semana 4 — Reporte Académico y Cierre del Proyecto

Objetivo: Redactar el reporte técnico en formato IEEE, documentar el código y producir el entregable final.

VI. COMPETENCIAS Y RESULTADOS DE APRENDIZAJE

VI-A. Competencias Técnicas (CT)

- CT-01** Implementar KNN desde cero con Python puro y NumPy.
- CT-02** Ejecutar un pipeline completo de ML supervisado (EDA → preprocesamiento → modelo → evaluación).

VI-B. Competencias Metodológicas (CM)

- CM-01** Aplicar el ciclo CRISP-DM en un proyecto de clasificación supervisada.
- CM-02** Redactar un reporte técnico en formato IEEE con estructura académica estándar.
- CM-03** Seleccionar y justificar métricas de evaluación según el contexto del problema.

VI-C. Competencias Transversales (CX)

- CX-01** Documentar código técnico con docstrings y READMEs profesionales.
- CX-02** Gestionar un repositorio Git con commits descriptivos y organización limpia.
- CX-03** Reflexionar críticamente sobre las limitaciones de un modelo y proponer mejoras.

VII. CRITERIOS DE EVALUACIÓN Y KPIs

Criterio de avance. Se considera completado el proyecto cuando se cumple al menos el **80 %** de los KPIs de la Tabla VIII y se han entregado todos los artefactos de la Sección VIII. No se avanza al Mes 02 sin cumplir este umbral.

VIII. ENTREGABLES DEL PROYECTO

- Notebook Jupyter:** EDA, implementación KNN manual, experimentos y comparación vs. sklearn. Celdas ejecutables sin errores.
- Curvas K vs. Accuracy:** sweep $K = 1$ a $K = 30$ con K^* identificado y justificado.
- ≥3 visualizaciones:** histogramas EDA, curva de validación, fronteras de decisión o matriz de confusión.

Cuadro VIII
RÚBRICA DE EVALUACIÓN — IA-ML-01

Criterio	Peso	KPI / Indicador de logro
KPIs numéricos	40 %	Accuracy $\geq 0,80$ en test set con implementación propia.
Reporte IEEE	25 %	≥ 6 secciones completas en formato IEEE (≥ 2 pp).
Código GitHub	20 %	Repositorio público con ≥ 3 commits semanales y README ejecutable.
Reflexión CausalBench	15 %	Párrafo de 150+ palabras con conexión explícita al roadmap.

4. **Reporte IEEE:** ≥ 2 páginas con Abstract, Introduction, Dataset, Methodology, Experiments, Results, Conclusions y References.

5. **Repositorio GitHub:** README instruccional, `requirements.txt`, estructura limpia, ≥ 3 commits semanales.

IX. FUNDAMENTOS MATEMÁTICOS DE KNN

IX-A. Métricas de Distancia

La elección de la métrica define el concepto de “vecindad” en el espacio de características [8]:

$$d_E(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (1)$$

$$d_M(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i| \quad (2)$$

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \quad (3)$$

donde $p = 2$ en (3) recupera (1) y $p = 1$ recupera (2).

IX-B. Regla de Clasificación

Dado un punto de consulta $\mathbf{x}_q$, la clase predicha es:

$$\hat{y} = \arg \max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} \mathbf{1}[y_i = c] \quad (4)$$

donde $\mathcal{N}_k(\mathbf{x}_q)$ denota el conjunto de los k vecinos más cercanos a $\mathbf{x}_q$.

IX-C. Impacto del Hiperparámetro K

- $K = 1$: máxima varianza (overfitting); frontera de decisión muy irregular.
- $K = n$: máximo sesgo (underfitting); predice siempre la clase mayoritaria.
- K^* óptimo: determinado empíricamente mediante validación sobre la curva Accuracy- K .

Cuadro IX
HERRAMIENTAS Y VERSIONES MÍNIMAS REQUERIDAS

Herramienta	Vers.	Uso en el proyecto
Python	3.10+	Lenguaje principal; implementación KNN manual.
NumPy	1.24+	Operaciones vectoriales; cálculo de distancias.
Pandas	2.0+	Carga de CSV; EDA; manejo de valores faltantes.
Matplotlib	3.7+	Histogramas, curvas K -accuracy, fronteras 2D.
scikit-learn	1.3+	Referencia: KNN, métricas, scalers, split.
Jupyter	7.0+	Entorno interactivo; narración científica.
Git/GitHub	—	Control de versiones; repositorio público.

Cuadro X
PLAN SEMANAL — 4 SEMANAS / ≈ 36 HORAS TOTALES

Semana	Actividad	Tareas y entregable
S1 (días 1–7)	Estudio teórico	Gérón Caps. 1–3; StatQuest KNN; glosario 20 términos. <i>Entrega: glosario técnico.</i>
S2 (días 8–14)	EDA + impl.	Carga UCI Weather; preprocesamiento; clase <code>KNNClassifier</code> . <i>Entrega: notebook con KNN funcional.</i>
S3 (días 15–21)	Experimentos	Sweep $K = 1, 30$; comparación sklearn; visualizaciones. <i>Entrega: notebook con ≥ 3 visualizaciones.</i>
S4 (días 22–28)	Escritura	Reporte IEEE ≥ 2 pp; limpieza de código; commit final. <i>Entrega: reporte PDF + GitHub completo.</i>

X. STACK TECNOLÓGICO

XI. CALENDARIO DEL PROYECTO

XII. CONEXIÓN CON CAUSALBENCH

CausalBench (Mes 12) es un benchmark que mide la pérdida de información causal en sistemas de memoria de agentes LLM [7]. La Tabla XI traza la continuidad de las competencias adquiridas en el Mes 01.

CONCLUSIÓN

El Mes 01 establece la base metodológica, técnica y de escritura académica que se amplifica progresivamente durante los 11 proyectos siguientes. Más allá de dominar KNN, el estudiante internaliza el *ciclo científico completo*: hipótesis $\rightarrow$ implementación $\rightarrow$ experimentación $\rightarrow$ análisis $\rightarrow$ comunicación, que es en esencia el mismo ciclo que subyace a la producción del paper CausalBench en el Mes 12.

Cuadro XI
TRAZABILIDAD MES 01 → CAUSALBENCH

Concepto (Mes 01)	Evol.	Aplicación en CausalBench
Pipeline EDA→modelo→eval	Meses 2-12	Marco metodológico base de CausalBench.
Métricas F1, Accuracy, CM	Mes 11	CausalRecall y CausalF1 son extensiones.
Normalización de features	Mes 09	Comparación de estrategias de memoria.
Escritura IEEE	Meses 2-12	Paper final en formato ACL/EMNLP.
Documentación GitHub	Todos	Dataset y código en HuggingFace (Mes 12).

REFERENCIAS

- [1] A. Gérón, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, 2022.
- [2] L. W. Anderson and D. R. Krathwohl (Eds.), *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.
- [3] UCI Machine Learning Repository, "Rain in Australia Dataset," <https://www.kaggle.com/datasets/jsphyg/weather-dataset-rattle-package>, 2019.
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [5] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conf. (SciPy 2010)*, pp. 56–61, 2010.
- [6] J. Starmer, "StatQuest: K-Nearest Neighbors, Clearly Explained," *YouTube — StatQuest with Josh Starmer*, <https://youtu.be/HVXimeOnQeI>, 2020.
- [7] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," *Trans. ACL*, vol. 12, pp. 157–173, 2024.
- [8] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [10] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. (draft). <https://web.stanford.edu/~jurafsky/slp3/>, 2023.